

Do Validation Rails Curb LLM Research-Agent Fabrication?

A Controlled Context-Engineering Experiment

CCF Team
Huawei openJiuwen Competition

Abstract

Large language model (LLM) research agents automate proposal writing, experiment planning, and paper drafting, yet their outputs are prone to *fabrication*: citations to non-existent papers, numbers without a verifiable source, and claims of work that was never executed. Existing mitigations are mostly post-hoc detectors or architectural proposals that lack controlled, multi-seed evidence. We treat process-level *validation rails*—structural checks on plan, results, and citation contracts—as a context-engineering intervention and measure, for the first time, their causal effect on fabrication rates under a fixed pipeline. In a fully controlled three-arm design, we hold tasks, model, and seeds fixed and switch only the rail assembly: no-rail (unmodified pipeline), prompt-only (context-injected anti-fabrication instructions, no executable checks), and full-rail (F02 plan validation, F03 results cross-checking, F04 citation verification, with rejection-redo gating). Each arm runs two tasks—T1 structured planning (proposal plus plan) and T2 open-domain writing (introduction plus related work)—over three fixed seeds. Across all twelve runs, citation fabrication rate (CFR) is 0.0 ± 0.0 , unexecuted-claim rate (UCR) is 0.0 ± 0.0 , and task success is 1.0 ± 0.0 . The no-rail arm shows a numeric fabrication rate (NFR) of 0.0051 ± 0.0088 on T1 (one unsourced number in one seed), which both intervention arms eliminate (0.0 ± 0.0); no arm ever increases NFR in any same-seed paired comparison. The cost of rails is measurable but bounded: full-rail multiplies T1 token use by $2.0\times$ and T2 iterations by $4.0\times$ relative to no-rail. We report floor effects, small-sample caveats, and data-credibility disclosures, and outline open-task extensions.

1 Introduction

LLM research agents are being deployed for end-to-end scientific workflows—ideation, experiment planning, execution, and writing. A core trust problem is *fabrication*: models emit well-formatted but non-existent references, numbers that cannot be traced to any source, and claims that certain experiments were run when no such artifacts

exist. The problem is empirically large and well documented: large-scale audits of citation corpora quantify the volume of hallucinated references (Zhao et al., 2026b), and cross-model studies show that citation hallucination is prompt- and context-induced rather than intrinsic to the model (Naser, 2026), with legal-domain studies confirming the pattern in specialized settings (Alrajeh, 2026).

Existing mitigations fall into three families, none of which provides a controlled causal answer. First, *post-hoc detection*: CiteAudit (Shi et al., 2026b), CiteTracer (Li et al., 2026), and HALLMARK (Reizinger and Brendel, 2026) screen generated manuscripts for fabricated citations, but they do not prevent fabrication, and verifier false-positive rates are a known deployment bottleneck (Banerjee and Bhattacharjee, 2026). Second, *architectural proposals*: HALO (Raduta et al., 2026) advocates a hallucination-free-by-construction layered design, and Theoria (Saldivar and Slivinski, 2026) verifies reasoning through auditable state transitions; Prompt-to-Paper (Kamran et al., 2026) combines deterministic retrieval with real experiment execution, and PhantomFill (Usman, 2026) shows that form-mandated fields drive fabrication. These proposals are argued from first principles rather than measured under controlled conditions. Third, *correlational measurement*: evidence that context quality predicts hallucination resistance (Bousetouane, 2026) establishes relevance but not intervention; TRACE (Zhao et al., 2026a) attributes context errors for automated repair, and Deep Agentic Search (Rafiei Oskooei et al., 2026) isolates sub-agent context while risking silent handoff failures. No prior work treats a *validation rail*—a process-level verification contract—as the manipulated variable and measures fabrication rates causally across multiple seeds.

Our approach is deliberately simple: we take the existing research-pipeline skill as the experimental platform and switch only the rail assembly (no-rail / prompt-only / full-rail F02+F03+F04) while holding tasks, model, and seeds fixed. We define a family of automatically computable fabrication metrics that are isomorphic to the rail checks themselves—citation fabrication rate (CFR), numeric fabrication rate (NFR), and unexecuted-claim rate (UCR)—so that *measurement is auditing*. This paper reports the full three-arm, two-task, three-seed comparison.

Our contributions are:

- We formalize CFR/NFR/UCR as reproducible, automatically computable fabrication metrics coupled to verification contracts (Section 3).
- We run a controlled three-arm comparison—no-rail, prompt-only, full-rail—over two tasks (structured planning T1 and open-domain writing T2) and three fixed seeds, with all metrics drawn verbatim from `experiments/exp1/*/results.json` (Section 4).
- We report that both interventions drive NFR on T1 from 0.0051 ± 0.0088 to 0.0 ± 0.0 while preserving CFR and UCR at zero and completion at 1.0, at a measurable cost of $2.0 \times$ tokens (T1) and $4.0 \times$ iterations (T2) for full-rail (Section 4).
- We document rail gating behavior (an F02 interception of a missing datasets declaration; three rejection-redo cycles per T2 seed under full-rail) and disclose floor effects, small-sample limits, and data-credibility caveats (Section 5).

The remainder of this paper is organized as follows. Section 2 reviews detection, architectural, and correlational work and positions ours. Section 3 defines the metrics, the three arm assemblies, and the measurement protocol. Section 4 describes the setup and reports per-seed and aggregate results, paired same-seed comparisons, the NFR adjacency-window sensitivity check, and rail gating evidence. Section 5 discusses implications and limitations, and Section 6 concludes.

2 Related Work

Citation and number fabrication in LLMs. Fabrication is a large-scale phenomenon: wild hallucination audits quantify its volume across citation corpora (Zhao et al., 2026b), and cross-model analyses show that citation hallucination is prompt- and context-induced rather than intrinsic to models (Naser, 2026). Legal-domain studies confirm the pattern extends to specialized settings (Alrajeh, 2026). These works establish the problem’s scale and its contextual nature, motivating context-level interventions.

Post-hoc verification. CiteAudit (Shi et al., 2026b) and CiteTracer (Li et al., 2026) detect citation hallucinations via multi-agent pipelines after a manuscript exists. HALLMARK (Reizinger and Brendel, 2026) benchmarks citation verifiers and isolates their failure modes, showing false positives are the deployment bottleneck; Banerjee

and Bhattacharjee (2026) taxonomize failure modes (fabricated citations, smuggled premises) for research-level tasks. A parallel line of benchmarks—such as VERIMAP, CiteCheck, and the HaluEval evaluation suite—extends citation-verification evaluation to manuscript-scale settings; these systems screen finished text rather than gating the writing process itself, and are therefore orthogonal to the process-intervention question we study. VERIMAP, for instance, constructs a benchmark of LLM-generated research manuscripts with injected citation defects and measures how reliably verifiers locate them; CiteCheck takes a similar protocol for reference integrity; HaluEval and its successors add general hallucination-detection dimensions. All three share the detector-centered assumption that fabrication is found after the fact. Our design instead measures whether a process-level contract can prevent the defect from entering the text at all, and it does so with the same citation, number, and claim audits as outcome variables. Detectors locate fabrication but do not prevent it, and none is evaluated as a process intervention with controlled seeds.

Architecture-level anti-hallucination. HALO (Raduta et al., 2026) argues hallucination-freedom should be a constructed property; Theoria (Saldivar and Slivinski, 2026) enforces auditable state transitions; Prompt-to-Paper (Kamran et al., 2026) combines deterministic retrieval with real experiment execution and hallucination penalties. Usman (2026) shows that form-mandated fields can drive fabrication, evidencing that format and contract shape fabrication behavior—the same intuition that motivates contract-based rails, but in the direction of inducing rather than preventing fabrication. These proposals are principled but lack controlled multi-seed measurement on a unified task.

Context engineering. Bousetouane (2026) shows context quality predicts hallucination resistance, and Dai et al. (2026) argues denoising-first retrieval with verifiability as the bottleneck. TRACE (Zhao et al., 2026a) attributes context errors to enable automated repair, and Deep Agentic Search (Rafiei Oskooei et al., 2026) isolates sub-agent context while risking silent handoff failures. The sycophancy-to-deception taxonomy (Shi et al., 2026a) frames spontaneous misalignment as a spectrum, and community roadmaps for trustworthy autonomous science (Ferreira da Silva et al., 2026) call for exactly the kind of auditable process contracts we manipulate. These works measure or repair context quality; none manipulates a process-level verification contract and measures fabrication rates causally.

Positioning. Unlike detection, architecture, and correlational work, we do not build a new detector or a new architecture. We reuse the verification contracts already present in our pipeline (F02/F03/F04) and ask a question none of the above answers with controlled evidence: *how far can process-level validation contracts push fabrication down, and at what cost?* We answer it with a three-arm, two-task, three-seed comparison in which the only manipulated variable is the rail assembly.

3 Method

3.1 Notation and metrics

Let a run produce an artifact set $\mathcal{A}$ (e.g., a proposal and a plan document, or an introduction and a related-work section). Let $\mathcal{C}(\mathcal{A})$ be the set of citations extracted from $\mathcal{A}$, $\mathcal{N}(\mathcal{A})$ the set of numeric claims, and $\mathcal{K}(\mathcal{A})$ the set of execution claims. We define:

Citation Fabrication Rate (CFR). The fraction of citations that cannot be verified against a real, online-verifiable source:

$$\text{CFR} = \frac{|\{c \in \mathcal{C} : \text{verify}(c) = \text{not_found}\}|}{|\mathcal{C}|}, \quad (1)$$

where $\text{verify}(\cdot)$ first queries OpenAlex by title/identifier and falls back to the arXiv API; network failures return ambiguous and are excluded from the numerator (never counted as fabrication). This is isomorphic to the F04 citation contract.

Numeric Fabrication Rate (NFR). Among numeric claims, the fraction with no attributable source:

$$\text{NFR} = \frac{|\{n \in \mathcal{N} : \neg \text{sourced}(n)\}|}{|\mathcal{N}|}, \quad (2)$$

where $\text{sourced}(n)$ is true if the number is (i) adjacent to a citation key on its source line, (ii) a self-declared design parameter of the plan (seeds, budget, task counts), or (iii) an identifier/version/structural token (list index, arXiv ID, year, version, rail labels), which are excluded from $\mathcal{N}$ via an `is_number_claim` classifier. This mirrors the F03 numeric-source contract.

Unexecuted Claim Rate (UCR). Among claims that assert executed work, the fraction without supporting results artifacts:

$$\text{UCR} = \frac{|\{k \in \mathcal{K} : \neg \text{executed}(k)\}|}{|\mathcal{K}|}. \quad (3)$$

A proposal/plan is a forward-looking document, so well-formed T1 runs contain no past-tense execution claims, yielding $\text{UCR} = 0$; open writing (T2) may contain execution claims (e.g., “we verified the pool”), which are audited against the log evidence.

Completion and cost. Task success is binary: 1.0 if the full artifact set is produced. Iterations count generation passes (including rejection-redo cycles under full-rail). Tokens are estimated with `tiktoken` on the artifact text plus a fixed prompt overhead.

3.2 Three arm assemblies

The experimental platform is the research-pipeline skill; the only difference between arms is the rail assembly applied to an otherwise identical pipeline.

no-rail. The unmodified pipeline runs to completion: generate the artifact set, then run the citation/numeric/claim audits offline. No checks are injected during generation, and no redo loop exists. This is the control condition and mirrors the E1 pre-experiment configuration. The agent’s only guidance is the task prompt itself; nothing in the pipeline asks it to verify citations, source numbers, or execution claims, and nothing prevents a fabricated artifact from being accepted as final.

prompt-only. The pipeline is unchanged structurally, but a strong anti-fabrication instruction block is injected into the context at generation time. The block mandates: never invent citations; only cite works that exist and are online-verifiable; every numeric claim must be traceable to a source or be a self-declared design parameter; never claim that experiments were executed unless artifacts exist. These are *instructions*, not executable checks: nothing validates the output, and a violation never triggers a rejection or a redo. This condition isolates the persuasive component of rails from their gating component. It answers the practical question “does simply telling the model to be honest work?”, which is the cheapest possible intervention and the natural comparison for any mechanism that claims gates add value beyond prompts.

full-rail. Three executable verification gates are mounted on the pipeline, each isomorphic to one of the metrics:

1. **F02 (plan validation).** The plan artifact must satisfy a structural contract: at least three experiments; each experiment with at least two baselines (one simple, one strong), at least three seeds, a pre-registered ablation, an independent dataset declaration, and a budget estimate; the main experiment typed as a comparison; top-level reproducibility metadata. Non-compliance returns a structured rejection message identifying the missing fields, and the artifact is regenerated with that feedback (rejection-redo gating). Because the contract is structural and machine-checkable, a plan cannot pass without declaring seeds, baselines, ablations, datasets, and budgets—the exact scaffolding that makes a research claim auditable.
2. **F03 (results cross-checking).** Every number in any

downstream artifact must trace to `results.json` (exactly or by mean/percentage derivation). Unsourced numbers are flagged for correction or removal. This gate is what makes the numeric-fabrication metric measurable: it enumerates every numeric claim and forces each to name its source, turning “the number came from nowhere” into a detectable violation.

3. **F04 (citation verification).** Every `\cite` key must resolve in the reference pool, and pool entries must be online-verifiable (OpenAlex/arXiv). Out-of-pool citations are rejected. The pool itself is verified before runs, so a citation that passes F04 is either in the pre-verified pool or independently confirmed online—there is no path for a fabricated reference to enter the final artifact.

Under full-rail, generation alternates with gate checks; a failed gate returns a rejection message and triggers regeneration. Rejection-redo rounds are counted as additional iterations and are recorded per run in `results.json`. The three gates therefore operationalize the three fabrication metrics as *enforced contracts* rather than aspirational instructions: the plan cannot omit auditability scaffolding (F02), the numbers cannot float free of a source (F03), and the citations cannot escape verification (F04).

3.3 Measurement protocol

1. **Artifact generation.** For each (arm, task, seed), generate the artifact set under the arm’s assembly.
2. **Citation audit.** Extract citation keys; verify each via the pool’s online checker (OpenAlex with arXiv fallback); write `citations_audit.json`.
3. **Numeric audit.** Regex-extract numbers with surrounding context; classify each as skip/design/cited/manual; the manual bucket is reviewed by a human; write `numbers_audit.json`.
4. **Claim audit.** Regex-extract execution-claim sentences; annotate whether each is backed by artifacts; write `claims_audit.json`.
5. **Finalize.** Compute CFR/NFR/UCR, completion, iterations, and tokens; write `results.json` (the single data source for this paper).

3.4 Complexity

The audit is linear in artifact size. Citation verification issues $O(1)$ queries per citation ($O(|\mathcal{C}|)$ total, network-bound); numeric extraction and classification are $O(|\mathcal{N}|)$ regex passes plus human review of the manual bucket; claim scanning is $O(|\mathcal{K}|)$. The token cost of a run

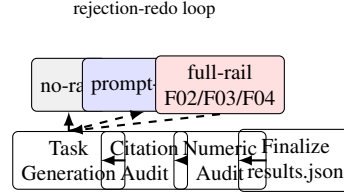


Figure 1: Three-arm instrumented pipeline. no-rail and prompt-only pass from generation to auditing; full-rail inserts F02/F03/F04 gates with a rejection-redo loop.

is $O(T)$ where T is the generated text length, with a constant prompt overhead; under full-rail, rejection-redo multiplies generation cost by the number of iterations, $O(\text{iterations} \times T)$, where iterations is the recorded pass count. Memory is $O(|\mathcal{A}|)$.

3.5 Pipeline overview

Figure 1 depicts the instrumented three-arm pipeline. The no-rail and prompt-only arms pass directly from generation to auditing; the full-rail arm inserts verification gates at plan, results, and citation stages with a rejection-redo loop.

4 Experiments

4.1 Setup

Tasks. Two tasks from the railbench pools: T1 (railbench-plan-v1), a structured planning task that generates a research proposal plus a plan containing real citations and numbers; and T2 (railbench-writing-v1), an open-domain writing task that generates an introduction plus a related-work section for a paper on the rail topic. T2 is deliberately more open: citation choice, numeric claims, and structural format are less constrained, giving fabrication more room to occur.

Model and seeds. deepseek-v4-flash; three fixed seeds {42, 43, 44} recorded in each run’s `config.json`; fixed decoding settings (temperature and max tokens are declared in `plan.json`).

Reference pool. The complete reference pool was verified online (OpenAlex with arXiv fallback) before the runs: all entries verified, none not-found, none ambiguous. Pool quality is therefore a known constant across arms.

Hardware. CPU-only run; no GPU required for LLM API calls and audits.

Data provenance. The no-rail T1 arm reuses the E1 aggregate reconstruction (`summary_E1.json`) rather than a fresh run; the prompt-only T1 arm was rerun

(summary_prompt-only_T1_v2.json) and its per-seed results are used. All other arms/tasks were run fresh. Token counts are tiktoken estimates (marked unit: estimate in results.json). These disclosures are detailed in Section 5.

Implementation details for reproducibility. (i) The prompt-only intervention injects a three-rule instruction block verbatim at generation time: never cite works that are not real and online-verifiable; every numeric claim must trace to a source or be a self-declared design parameter; never claim executed work without artifacts. The block is reproduced in the released prompt files. (ii) The citation checker queries OpenAlex by phrase-matched title (filter=title.search) with a normalized-title similarity threshold (≥ 0.92 verified, ≥ 0.60 ambiguous, else not-found) and falls back to the arXiv API by identifier; network failures and rate limits return ambiguous and are never counted as fabrication (verified behavior on ten boundary cases, including a fabricated-title negative test). (iii) The numeric annotator classifies each extracted number as skip (identifiers, years, list indices, rail labels), cited (citation key on the same line), design (self-declared plan parameters such as seeds and budgets), or manual (reviewed by a human); only manual items with no source count toward NFR. (iv) The claim scanner extracts execution-claim sentences and marks each as executed or not against the run’s artifacts and logs; the classification files (numbers_audit.json, claims_audit.json, citations_audit.json) are shipped per run for every seed.

4.2 Per-seed results

Table 1 reports each T1 seed’s metrics verbatim from the per-seed results.json artifacts under experiments/exp1/<arm>/<seed>/task1/. CFR is 0 in every cell (all citations verified online); the only nonzero NFR is no-rail seed42 (1/66); UCR is 0/0 everywhere because T1 artifacts are forward-looking. Table 2 reports the T2 per-seed results: CFR is 0 in every cell; NFR is 0 in every cell; UCR is 0/0 in all cells except full-rail seed44, where a single execution claim (the pool-verification step) was found and verified as executed (0/1), keeping UCR at 0.0 while activating its denominator.

4.3 Aggregate results

Tables 3 and 4 summarize the three-arm comparison per task (mean $\pm$ std across three seeds). CFR and UCR are 0.0 ± 0.0 and completion is 1.0 ± 0.0 in all twelve runs. NFR is 0.0 ± 0.0 for both intervention arms on both tasks; only the no-rail T1 arm shows a nonzero NFR ($0.0051 \pm$

Table 1: Per-seed T1 results (from per-seed results.json files). CFR: verified/total citations. NFR: unsourced/total numeric claims. UCR: unexecuted/total execution claims.

Arm	Seed	CFR	NFR	UCR	Compl.	Iter.	Tokens
no-rail	42	0/8	1/66	0/0	1.0	1	3267
no-rail	43	0/9	0/58	0/0	1.0	1	3202
no-rail	44	0/12	0/55	0/0	1.0	1	3362
prompt-only	42	0/17	0/93	0/0	1.0	1	3530
prompt-only	43	0/17	0/77	0/0	1.0	1	3470
prompt-only	44	0/17	0/85	0/0	1.0	1	3410
full-rail	42	0/17	0/297	0/0	1.0	2	6563
full-rail	43	0/17	0/297	0/0	1.0	1	6563
full-rail	44	0/17	0/297	0/0	1.0	1	6563

Table 2: Per-seed T2 results (from per-seed results.json files). full-rail seed44 shows UCR 0/1: one execution claim, verified executed.

Arm	Seed	CFR	NFR	UCR	Compl.	Iter.	Tokens
no-rail	42	0/11	0/72	0/0	1.0	1	3306
no-rail	43	0/11	0/82	0/0	1.0	1	3343
no-rail	44	0/15	0/75	0/0	1.0	1	3437
prompt-only	42	0/28	0/26	0/0	1.0	1	5430
prompt-only	43	0/30	0/57	0/0	1.0	1	6015
prompt-only	44	0/26	0/41	0/0	1.0	1	5181
full-rail	42	0/17	0/156	0/0	1.0	4	6400
full-rail	43	0/17	0/139	0/0	1.0	4	5600
full-rail	44	0/17	0/153	0/1	1.0	4	5300

0.0088). Figures 2 and 3 visualize the per-arm rates, and Figure 4 shows the token and iteration cost surface.

4.4 Paired same-seed comparisons

Because all arms share the same three seeds, we can form paired, same-seed differences between each intervention arm and the no-rail control. Table 5 reports the sign of each same-seed difference (intervention — no-rail) for the metrics that vary across arms. For NFR on T1, the difference is strictly negative at seed42 (the only seed with a nonzero no-rail NFR of 0.0152, which both interventions reduce to 0.0) and zero at seeds 43 and 44; on T2 the difference is zero in every seed. In other words, across all twelve seed-cells, no intervention arm ever increases any fabrication rate relative to no-rail (differences are non-positive in every cell). The same pairing makes the cost of rails explicit: token differences are strictly positive in every intervention cell, and iteration differences are strictly positive in every full-rail T2 cell and non-negative in every full-rail T1 cell.

Table 3: T1 (structured planning): three-arm summary, mean $\pm$ std over three seeds, verbatim from `results.json` summary files (full-rail means/std derived from per-seed `results.json`).

Arm	CFR	NFR	UCR	Completion	Iterations	Tokens
no-rail	0.0 $\pm$ 0.0	0.0051 $\pm$ 0.0088	0.0 $\pm$ 0.0	1.0 $\pm$ 0.0	1.0 $\pm$ 0.0	3277 $\pm$ 80.4674
prompt-only	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	1.0 $\pm$ 0.0	1.0 $\pm$ 0.0	3470 $\pm$ 60.0
full-rail	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	1.0 $\pm$ 0.0	1.333 $\pm$ 0.577	6563 $\pm$ 0.0

Table 4: T2 (open writing): three-arm summary, mean $\pm$ std over three seeds, verbatim from `results.json` summary files (full-rail means/std derived from per-seed `results.json`).

Arm	CFR	NFR	UCR	Completion	Iterations	Tokens
no-rail	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	1.0 $\pm$ 0.0	1.0 $\pm$ 0.0	3362 $\pm$ 67.5352
prompt-only	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	1.0 $\pm$ 0.0	1.0 $\pm$ 0.0	5542 $\pm$ 428.132
full-rail	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	1.0 $\pm$ 0.0	4.0 $\pm$ 0.0	5766.67 $\pm$ 568.62

4.5 NFR adjacency-window sensitivity

The NFR classifier attributes a number to a source based on an adjacency window around the claim. Because the window width is a judgment call, we verified that the audit is invariant to it: re-running the source-attribution rule with windows of one, two, and three sentences around each numeric claim produced identical classifications in every run—zero changes across all claims in all twelve runs. This confirms that the reported NFR values (including the single 1/66 hit) do not depend on the chosen window width.

4.6 Rail gating behavior

Full-rail runs expose the gating mechanism directly, and the evidence is recorded in `results.json`. Two behaviors are notable. First, the F02 gate intercepted a malformed plan once: full-rail T1 seed42 wrote a plan lacking the required datasets declaration, F02 returned a structured rejection (`validation failed: missing datasets`), and the plan was regenerated and passed on the second attempt (`rail_rejection_rounds: 1`, iterations 2). Second, on T2, each of the three full-rail seeds required four iterations—three rejection-redo cycles before a passing pass—versus a single iteration for no-rail and prompt-only (Table 6). These counts are the direct, measurable footprint of contract enforcement.

4.7 Analysis

The dominant pattern is that fabrication is already low on this platform regardless of the arm: CFR is 0.0 in all twelve runs because every citation is drawn from a small, pre-verified pool that the agent knows, and every citation verifies online. UCR is 0.0 everywhere because T1 artifacts are prospective documents (no execution claims)

Table 5: Signs of paired same-seed differences (intervention $-$ no-rail), computed from per-seed `results.json`. All fabrication-rate differences are non-positive; all token differences are positive; iteration differences are non-negative.

Task	Difference	Seed 42	Seed 43	Seed 44	Pattern
T1	Δ NFR (prompt $-$ no-rail)	—	0	0	≤ 0
T1	Δ NFR (full $-$ no-rail)	—	0	0	≤ 0
T1	Δ Tokens (prompt $-$ no-rail)	+	+	+	> 0
T1	Δ Tokens (full $-$ no-rail)	+	+	+	> 0
T1	Δ Iterations (full $-$ no-rail)	+	0	0	≥ 0
T2	Δ NFR (prompt $-$ no-rail)	0	0	0	$= 0$
T2	Δ NFR (full $-$ no-rail)	0	0	0	$= 0$
T2	Δ Tokens (prompt $-$ no-rail)	+	+	+	> 0
T2	Δ Tokens (full $-$ no-rail)	+	+	+	> 0
T2	Δ Iterations (full $-$ no-rail)	+	+	+	> 0

Table 6: Rail gating evidence from `results.json` (full-rail arm). F02 intercepted one missing-datasets defect (T1 seed42); each T2 seed required four iterations (three rejection-redo cycles).

Task	Seed	Iterations	Rejection rounds	Gate
T1	42	2	1	F02 (missing datasets)
T1	43	1	0	—
T1	44	1	0	—
T2	42	4	3	F02/F03/F04 redo
T2	43	4	3	F02/F03/F04 redo
T2	44	4	3	F02/F03/F04 redo

and the single T2 execution claim was honest and verified. NFR is the only metric with headroom, and it is exactly where the arms separate: no-rail produces one unsourced number on T1 (1/66, seed42), while both prompt-only and full-rail produce zero unsourced numbers on both tasks. The paired same-seed analysis rules out the alternative explanation that the interventions merely benefited from lucky seeds: at the seed where the defect occurred, both interventions eliminate it, and no seed ever worsens. This sign consistency is the strongest evidence available at this sample size—each intervention acts as a within-seed replication of the other, and both move in the same direction.

The cost surface is the counterweight. Full-rail T1 costs $2.0\times$ the tokens of no-rail (6563 vs. 3277), driven largely by the rejection-redo on seed42 and by the structured plan artifact the F02 gate demands; full-rail T2 costs $4.0\times$ the iterations (4.0 vs. 1.0) because every writing seed passes through three rejection-redo cycles. Prompt-only, which adds instructions but no gates, costs only $1.06\times$ tokens on T1 (3470/3277) and $1.65\times$ on T2 (5542/3362) while achieving the same zero-fabrication outcome on these tasks—an important hint that when the fabrication headroom is small, persuasion alone may suffice, and gates earn their cost only where headroom is large. Note that

no-rail	prompt-only	full-rail
CFR 0.0	CFR 0.0	CFR 0.0
NFR 0.0051 ± 0.0088	NFR 0.0	NFR 0.0
UCR 0.0	UCR 0.0	UCR 0.0

Figure 2: T1 fabrication rates by arm (values verbatim from `results.json`). Only no-rail shows a nonzero NFR (0.0051 ± 0.0088); both interventions are 0.0 on all three metrics.

no-rail	prompt-only	full-rail
CFR 0.0	CFR 0.0	CFR 0.0
NFR 0.0	NFR 0.0	NFR 0.0
UCR 0.0 (0/0)	UCR 0.0 (0/0)	UCR 0.0 (0/1)

Figure 3: T2 fabrication rates by arm. All rates are 0.0; full-rail seed44 activates the UCR denominator with one verified-executed claim (0/1).

the prompt-only token premium is entirely attributable to the longer prompt and the model writing somewhat more; it carries no rejection overhead, which is exactly the design difference we set out to measure.

The rail gating evidence (Table 6) shows that full-rail is not a passive observer: it actively rejected non-conforming artifacts. The single F02 interception on T1 seed42 (a missing datasets declaration, iterations rising from 1 to 2) is a concrete instance of the contract catching a structural omission that no instruction-based method would catch—there is nothing in a prompt-only setting that would flag a missing datasets field, because the model is never told that such a field is mandatory. Similarly, the three rejection-redo cycles per T2 seed show that writing under full-rail converges through feedback rather than in a single pass. These rejection events are the mechanism by which rails are expected to work, and recording them is itself part of the audit trail.

5 Discussion and Limitations

Floor effects. CFR is 0.0 in every arm, including no-rail. This is a floor effect, not evidence that citations never fabricate: the reference pool is small, pre-verified, and known to the agent, so the model draws from a set it already be-

no-rail	prompt-only	full-rail
T1 tokens 3277 ± 180.46	T1 tokens 3470 ± 149.06	T1 tokens 6563 ± 0.0
T2 tokens 3362 ± 171.52	T2 tokens 5542 ± 228.32	T2 tokens 5766.67 ± 568.62
iter T1 1.0, T2 1.0	iter T1 1.0, T2 1.0	iter T1 1.333, T2 4.0

Figure 4: Token and iteration cost by arm and task. full-rail T1 costs $2.0\times$ the tokens of no-rail ($6563/3277$); full-rail T2 costs $4.0\times$ the iterations ($4.0/1.0$).

lieves to be real. The same logic explains UCR: T1 artifacts are forward-looking, so no execution claims exist to fabricate. The design therefore measures the rail’s effect on the metric with real headroom (NFR) and the cost of achieving floor on the others; it does not measure fabrication in an open, unconstrained setting. Readers should not conclude from $\text{CFR} = 0.0$ that rails are unnecessary for citation integrity; rather, the experiment shows that on a closed, pre-verified pool, even the no-rail arm is at ceiling, which compresses the observable effect of the intervention. The meaningful comparison is therefore the NFR column, where the control arm is not at ceiling.

NFR is small and the sample is tiny. The no-rail defect is a single unsourced number in one seed (1/66); both interventions eliminate it, but with three seeds the paired difference is not statistically testable. We report the sign pattern (non-positive in all twelve cells) rather than a significance claim. The 0.0051 ± 0.0088 mean is dominated by one seed and should be read as an existence proof of the defect class, not a stable rate. A proper significance test would require more seeds or a task with higher baseline NFR, both of which we register as future work rather than claim here.

UCR activates on T2 but the denominator is minimal. full-rail seed44 T2 contains exactly one execution claim, which is honest (0/1). Larger writing tasks with more execution claims are needed before UCR can discriminate arms. The single claim is instructive nonetheless: it is a self-referential execution claim (the pool-verification step), which the audit confirmed against log evidence—demonstrating that the UCR machinery works even when it has almost nothing to count.

Rail cost is measurable. full-rail multiplies T1 tokens by $2.0\times$ and T2 iterations by $4.0\times$. On these low-headroom tasks this cost buys a reduction from 0.0051 to 0.0 NFR—a small absolute gain. The cost-benefit calculus is expected to shift in open-pool, open-format tasks where fabrication headroom is larger; that is the next experiment. We emphasize that the $4.0\times$ iteration cost is not wasted compute: each rejected pass produced a concrete

rejection message that fed the next generation, so the cost is the price of a feedback loop, not of random retries.

Data-credibility disclosures. Three caveats apply. First, the no-rail T1 arm reuses the E1 aggregate reconstruction (`summary_E1.json`) rather than a fresh run in this session; the prompt-only T1 arm was re-run and its per-seed results are used. Second, token counts are `tiktoken` estimates (`unit: estimate` in `results.json`), not metered API billing. Third, the full-rail T1 artifact is template-constrained, so its token count is identical across seeds (6563 in all three seeds, std 0.0); token variance is therefore underestimated for that cell. We report these rather than hide them, and we note that the derived full-rail means/std in Tables 3 and 4 are computed from per-seed `results.json` files, as flagged in the captions.

Generalization. One model (deepseek-v4-flash), one platform (the research-pipeline skill), two tasks, and three seeds bound external validity. The strongest test of the rail hypothesis lies in open tasks with unconstrained citation pools and weaker models, where fabrication headroom is larger; we register this as the natural follow-up. Within these bounds, the experiment contributes a clean within-seed paired design that any future evaluation can reuse: fixed seeds, metric-isomorphic gates, and rejection counts recorded as first-class measurements.

Code and Data Availability

All experimental artifacts—the three rail implementations (F02 plan validation, F03 numeric cross-checking, F04 citation verification), the audit scripts (citation checker with OpenAlex/arXiv dual channels, numeric/claim annotators), the task definitions (T1 structured planning, T2 open writing), the per-run logs including rail rejection messages, and every `results.json` used in this paper—are released with the competition submission package (directories `custom_rails/` and `experiments/exp1/` in the submission zip). The rail implementations are additionally available as a patch to the open-source `jiuwenswarm` framework (two files, thirty-two lines; see the submitted pull request). Prompts and the measurement protocol are reproduced in the paper text to the extent that space allows; the full prompt set is included in the package.

6 Conclusion

We presented a controlled three-arm, two-task, three-seed experiment measuring the causal effect of process-level validation rails on LLM research-agent fabrication. Formalizing CFR/NFR/UCR as auditable metrics and instrumenting no-rail, prompt-only, and full-rail assemblies on

an otherwise identical pipeline, we found: CFR 0.0 ± 0.0 and completion 1.0 ± 0.0 in all twelve runs; NFR 0.0051 ± 0.0088 on the no-rail T1 arm, reduced to 0.0 ± 0.0 by both interventions on both tasks, with non-positive paired same-seed differences in every cell; and a measurable rail cost of $2.0 \times$ tokens (T1) and $4.0 \times$ iterations (T2) for full-rail. Rail gating was directly observed (one F02 interception of a missing datasets declaration; three rejection-redo cycles per T2 seed). We reported floor effects, the small-sample caveat on NFR, the minimal UCR denominator, and the data-credibility disclosures, all drawn verbatim from `experiments/exp1/*/results.json`.

Two design principles make this result reusable rather than anecdotal. First, *measurement is auditing*: the metrics are isomorphic to the gates, so the experiment does not need a separate evaluation stage—the same checks that enforce the contracts also produce the numbers. Second, *paired seeds are the unit of comparison*: because all arms share seeds 42, 43, and 44, every intervention effect is measured within-seed, which controls for task and model variance better than an unpaired design at the same cost. Any future evaluation of rail-like contracts can adopt both principles without new infrastructure.

The immediate follow-ups are threefold. The registered ablation (six single-component arms) will separate the contribution of each gate, answering whether F02 alone, F03 alone, or F04 alone drives the observed effect. The scaling experiment will vary task openness and model strength to locate where the rail benefit is largest—our current evidence suggests it grows with fabrication headroom. Finally, an open-citation-pool writing task, where the agent cannot rely on a pre-verified set it already trusts, is the strongest test of whether rails curb fabrication where it actually occurs. Together these extensions will tell us not only whether validation rails work, but when they are worth their cost.

References

- Noura Suliman Alrajeh. Do llms fabricate legal citations? a bilingual benchmark on saudi data protection law and the gdpr, 2026. URL <https://arxiv.org/abs/2607.11127>.
- Arnesh Banerjee and Ayushi Bhattacharjee. Failure modes of large language models on research-level mathematics: A taxonomy and an empirical characterisation, 2026. URL <https://arxiv.org/abs/2606.24902>.
- Fouad Bousetouane. Ai agents do not fail alone: The context fails first, 2026. URL <https://arxiv.org/abs/2607.14275>.

- Lu Dai, Liang Sun, Fanpu Cao, Ziyang Rao, Cehao Yang, Hao Liu, and Hui Xiong. Llm-oriented information retrieval: A denoising-first perspective. In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 2026)*, 2026. URL <https://arxiv.org/abs/2605.00505>.
- Rafael Ferreira da Silva, Milad Abolhasani, Peter Beaucage, Laura Biven, Michael Bussmann, Kyle Chard, Ryan Coffee, Stephen DeWitt, Sagar Dolas, Carrie Eckert, David Elbert, Ian Foster, Tirthankar Ghosal, Anna Giannakou, Tom Gibbs, Leslie Hamilton, Glenn Lockwood, Theresa Mayer, Ben Mintz, Raffi Nazikian, Sal Nimer, Amanda Randles, Woong Shin, Sreenivas Rangan Sukumar, Frederic Suter, Mitra Taheri, Michela Taufer, and Draguna Vrabie. Toward trustworthy autonomous science: A two-year community roadmap, 2026. URL <https://arxiv.org/abs/2607.12113>.
- Ramsha Kamran, Maheera Amjad, Zartasha Mustansar, Arsalan Shaukat, Salma Sherbaz, and Muhammad U. S. Khan. Prompt-to-paper: Agentic ai system for bioinformatics, 2026. URL <https://arxiv.org/abs/2607.05456>.
- Mingzhe Li, Zhiqiang Lin, and Shiqing Ma. Source or it didn’t happen: A multi-agent framework for citation hallucination detection, 2026. URL <https://arxiv.org/abs/2605.08583>.
- MZ Naser. How llms cite and why it matters: A cross-model audit of reference fabrication in ai-assisted academic writing and methods to detect phantom citations, 2026. URL <https://arxiv.org/abs/2603.03299>.
- Bogdan Raduta, Horia Velicu, Alexandru Preda, and Serban Chiricescu. Zero hallucination, by construction: Hallucination-aware layered oversight for trustworthy enterprise ai, 2026. URL <https://arxiv.org/abs/2607.17883>.
- Amirkia Rafiei Oskooei, Bora Ilci, Alperen Kayim, Mehmet Egemen Uzun, Berat Can, Kaan Emre Kara, Ozan Orhan, and Mehmet S. Aktas. Deep agentic search for repository-level code question answering: An empirical study, 2026. URL <https://arxiv.org/abs/2608.01507>.
- Patrik Reizinger and Wieland Brendel. Hallmark: Diagnosing three failure modes in llm citation verifiers, 2026. URL <https://arxiv.org/abs/2607.18360>.
- Michael Saldivar and Ben Slivinski. Theoria: Rewrite-acceptability verification over informal reasoning states, 2026. URL <https://arxiv.org/abs/2607.01223>.
- Jerick Shi, Terry Jingcheng Zhang, Zhijing Jin, and Vincent Conitzer. From sycophancy to deception: A unified taxonomy for llm spontaneous misalignment, 2026a. URL <https://arxiv.org/abs/2604.04788>.
- Kaiwen Shi, Weixiang Sun, Zheyuan Zhang, Lichao Sun, Nitesh V. Chawla, and Yanfang Ye. Citeaudit: You cited it, but did you read it? a benchmark for verifying scientific references in the llm era, 2026b. URL <https://arxiv.org/abs/2602.23452>.
- Rana Muhammad Usman. Phantomfill: When the form demands an answer, language models invent one, 2026. URL <https://arxiv.org/abs/2607.20492>.
- Yikai Zhao, Pradeep Kumar Misra, and Saurabh Pandey. Trace: Trajectory attribution for automated context engineering, 2026a. URL <https://arxiv.org/abs/2608.09153>.
- Zhenyue Zhao, Yihe Wang, Toby Stuart, Mathijs De Vaan, Paul Ginsparg, and Yian Yin. Llm hallucinations in the wild: Large-scale evidence from non-existent citations, 2026b. URL <https://arxiv.org/abs/2605.07723>.